

Automatically Deriving Cost Models for Structured Parallel Programs using Types and Hylomorphisms

David Castro, **Kevin Hammond** and Susmit Sarkar

University of St Andrews, UK



T: *@rephrase_eu, @khstandrews*

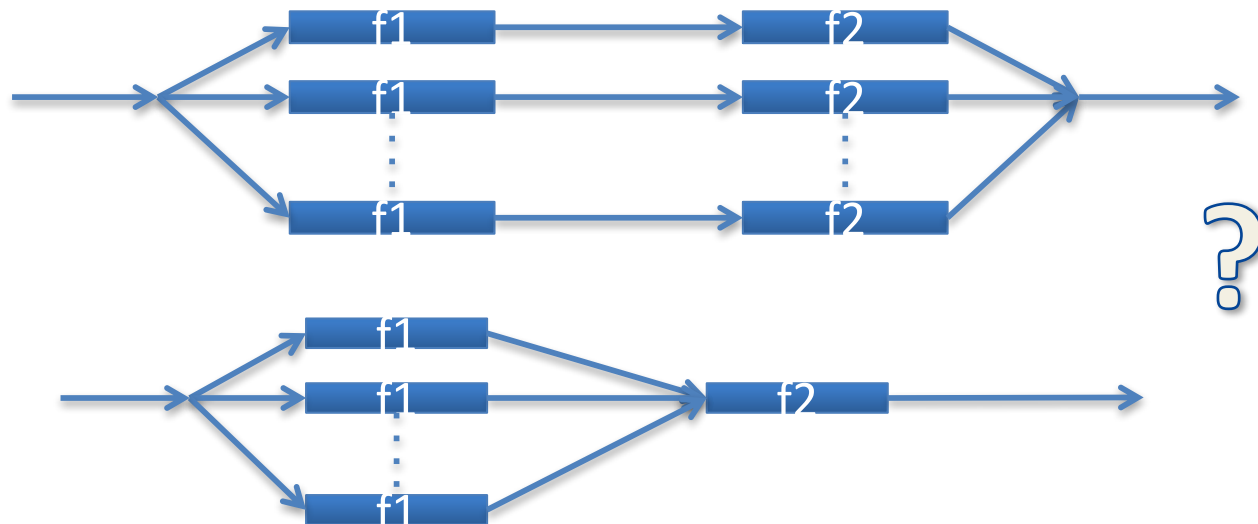
E: *kevin@kevinhammond.net*

W: <http://www.rephrase-ict.eu>
<http://www.paraformance.com>



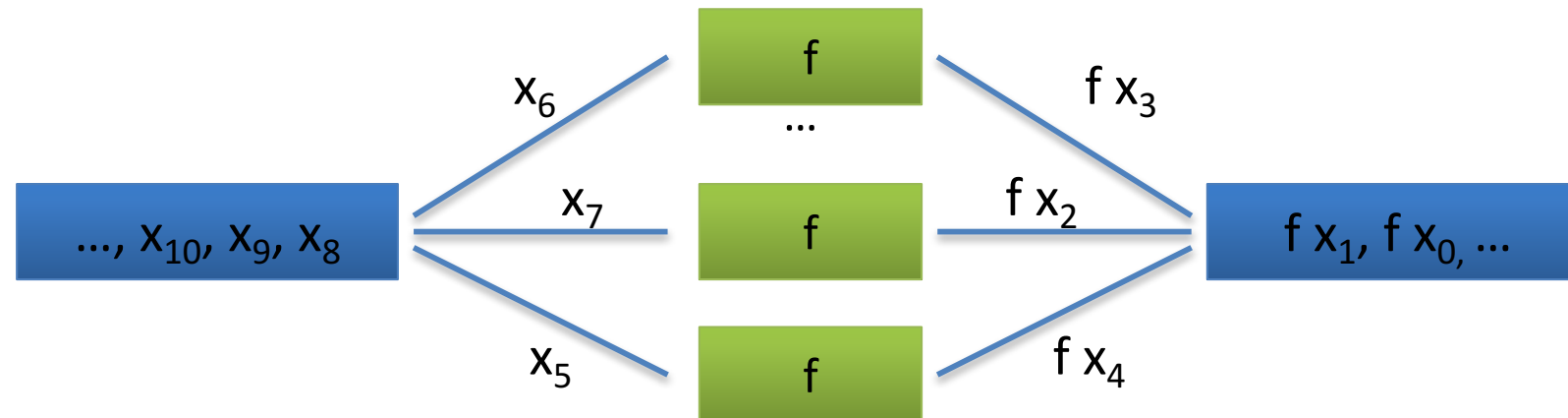
Motivation

- Parallel patterns are great
 - **BUT we need to choose the best implementation**
 - *For a specific (heterogeneous) parallel architecture*
- We need a way to reason about parallel **structure**
 - ✓ Correctness of transformations (done! **ICFP2016**)
 - ❑ Reasoning about performance (in progress)



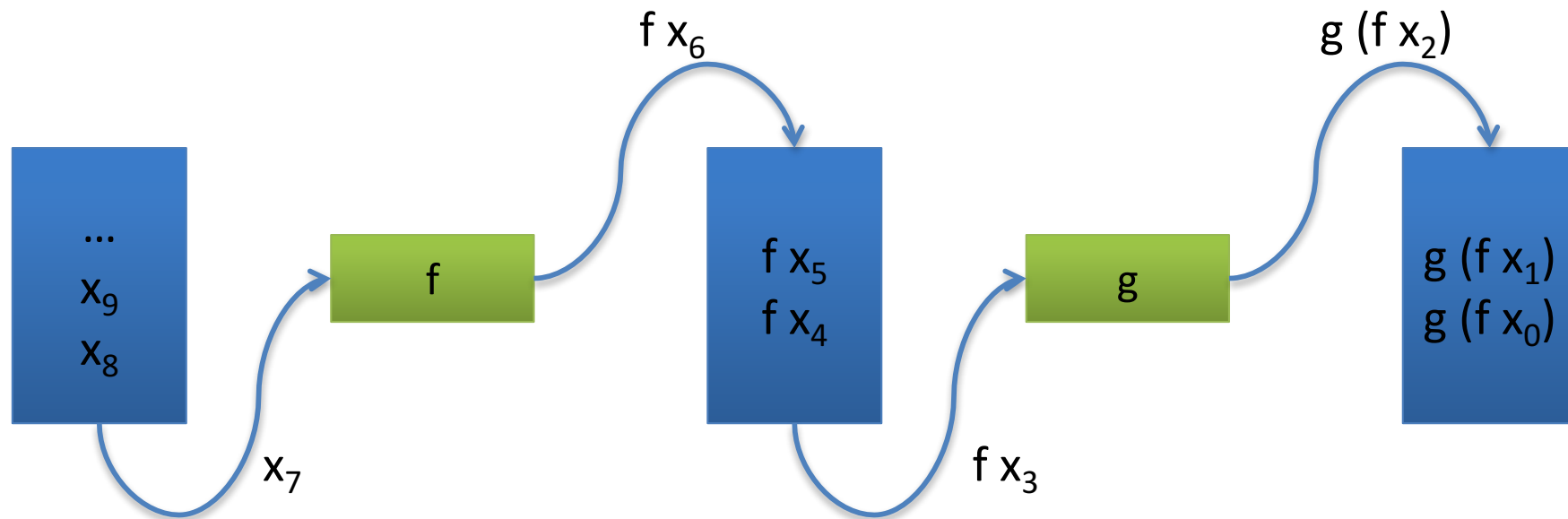
Example Skeleton: Parallel Task Farms

- Task Farms use a fixed number of workers (farm n f)
 - Each worker applies the same operation (f)
 - f is applied to each of the inputs in a stream.



Example Skeleton: Parallel Pipeline

- Parallel pipelines compose two operations ($f \parallel g$)
 - over the elements of an input stream
 - f and g are run in parallel



Example: Parallel Image Merge

Image merge (**im**) composes **mark** and **merge**

```
im : List (Img, Img) -> List Img  
im = map (merge ◦ mark)
```

There are many alternative parallel implementations
- **even just** using farms (**farm**) and pipelines (**||**)

```
im1 = farm n (fun (merge ◦ mark))  
im2 = farm n (fun mark) || farm m (fun merge)  
im3 = farm n (fun mark) || fun merge  
...
```

So, why types?

- According to the types community:
 - **Soundness:** “Well-typed programs cannot go wrong”
 - **Documentation:** “Type signatures provide valuable docs.”
- The benefits we are really interested in:
 - **Soundness:**
 - “Well-typed programs can be parallelised as described by the types”
 - **Documentation:**
 - “Type-level parallel structures clearly separate structure & functionality”
 - **Reusing well-understood techniques.**
 - E.g. algorithms for type unification and inference.

Selecting an Implementation

Decorate the function type with $IM(n,m)$

```
im :  $IM(n,m)$  ~ List (Img, Img) -> List Img  
im = map (merge ° mark)
```

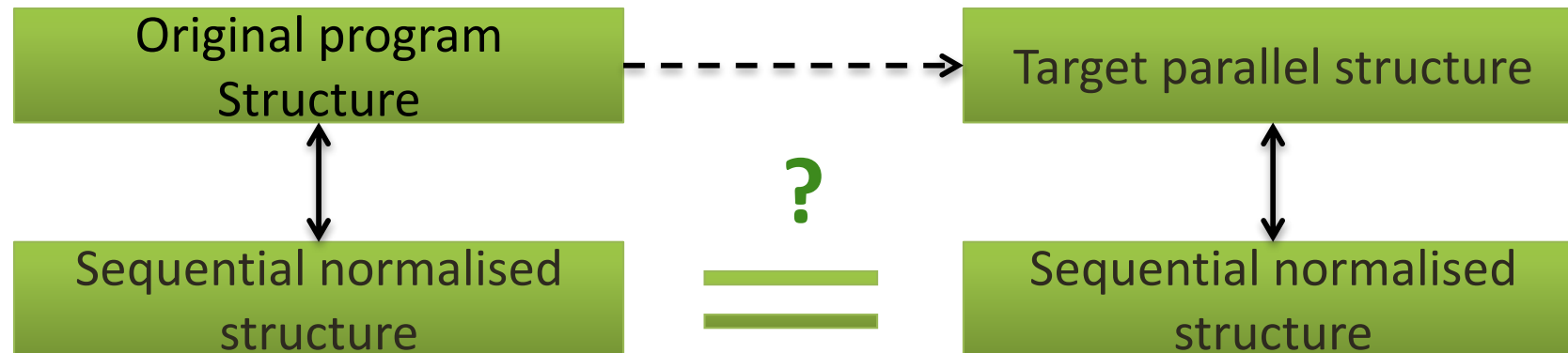
```
 $IM(n,m) = FARM\ n\ (FUN\ A) \parallel FARM\ m\ (FUN\ A)$ 
```

The type system now automatically selects

```
im2 = farm n (fun mark) || farm m (fun merge)
```

We can *guarantee* that this is functionally equivalent to *im*

Introducing/Transforming Parallel Patterns



How do we decide semantic equivalences?

- We can use the laws and properties of *hylomorphisms*!
- Hylomorphisms are a generalisation of a divide and conquer.

$$\text{hylo}_F g h = f$$

where $f = g \circ F f \circ h$

- “h” splits the input into a structure “F”, then recursive calls are mapped in structure “F”, the results are combined by “g”.
- Algorithmic skeletons can be described as instances of hylomorphisms

Hylomorphism Example

```
type T A = Empty | (A, List A, List A)
```

```
quicksort : List A -> List A
```

```
quicksort = hyloT merge split
```

```
merge : T A -> List A
```

```
merge = ...
```

```
split : List A -> T A
```

```
split = ...
```

Introducing Parallelism

We start with a streaming sequential version

```
quicksort : List (List A) -> List (List A)  
quicksort = mapList (hyloT merge split)
```

To create a **task farm** and **pipeline** version, just change the type!

```
quicksort : PARL (FARM n _ || _) ~  
List (List A) -> List (List A)
```

To create a parallel **divide-and-conquer** version, change the type again!

```
quicksort : PARL (DCn,T A A) ~  
List (List A) -> List (List A)
```

Base Semantics

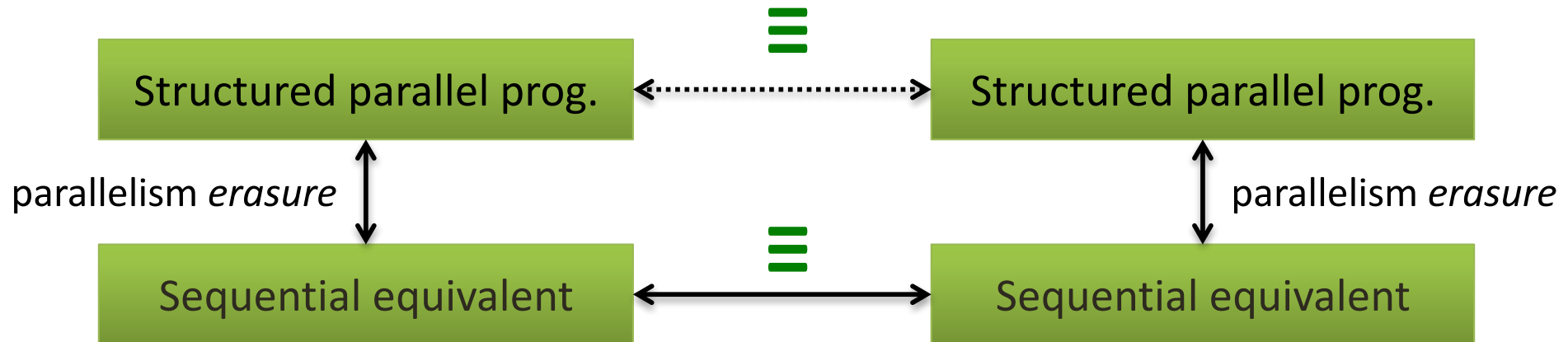
- Defined using well-known **recursion schemes**:
 - **map** (replication, map_F)
 - **fold** (“catamorphism”, cata_F)
 - **unfold** (“anamorphism”, ana_F)
- plus sequential composition, $\circ$

$$\begin{aligned}
 S[p : T A \rightarrow T B] &: [A \rightarrow B] \\
 S[\text{fun}_T f] &= \text{env}(f) \\
 S[p_1 \parallel p_2] &= S[p_2] \circ S[p_1] \\
 S[\text{farm } n \ p] &= S[p] \\
 S[\text{dc}_{n,T,F} f \ g] &= \text{cata}_F (\text{env}(f)) \circ \text{ana}_F (\text{env}(g))
 \end{aligned}$$

$$\begin{aligned}
 [p : T A \rightarrow T B] &: [T A \rightarrow T B] \\
 [p] &= \text{map}_T S[p]
 \end{aligned}$$

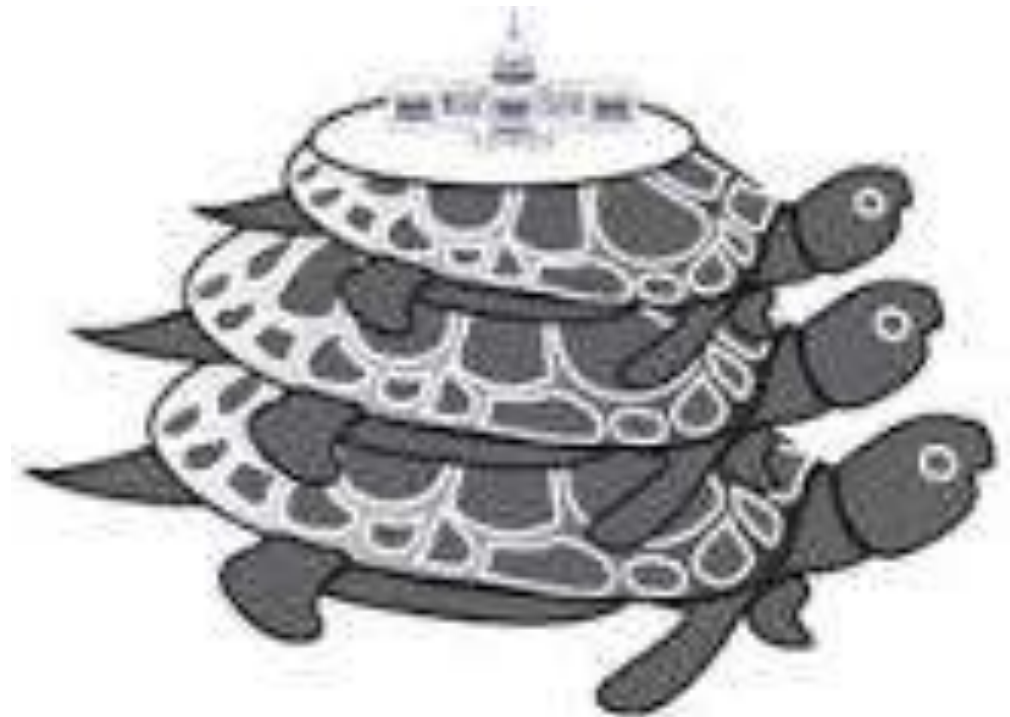
Deciding Semantic Equivalences

- Equivalence of parallel programs is reduced to equivalence of **recursion schemes**.



Recursion Schemes are Hylomorphisms!

$$\begin{aligned}T A &= \mu(F A) \\ \text{map}_T f &= \text{hylo}_{F A} (\text{in}_{F B} \circ (F f \text{ id})) \text{ out}_{F A}, \\ &\quad \text{where } A = \text{dom}(f) \text{ and } B = \text{codom}(f) \\ \text{cata}_F f &= \text{hylo}_F f \text{ out}_F \\ \text{ana}_F f &= \text{hylo}_F \text{ in}_F f\end{aligned}$$



Inferring Parallel Structures

We can leave holes in the types

$$\text{IM}(n,m) = _ \parallel \text{FARM } m _$$

Type unification replaces $_$ with any suitable parallel structure

$$\text{IM}(n,m) = \text{min cost } (_ \parallel \text{FARM } m _)$$

Type unification replaces $_$ with the parallel structure that *minimises* the cost model!

Example: Cost Model for Task Farms

$$\text{qfarm}(n, \mathcal{P})(Q_0, Q_1) = \overbrace{\mathcal{P}(Q_0, Q_1) \parallel \dots \parallel \mathcal{P}(Q_0, Q_1)}^{n \text{ times}}$$



This cost depends on
the number of
contending threads

If $\mathcal{P}$ takes time $\mathcal{T}$, then the cost of each $\mathcal{P}(Q_0, Q_1)$ is

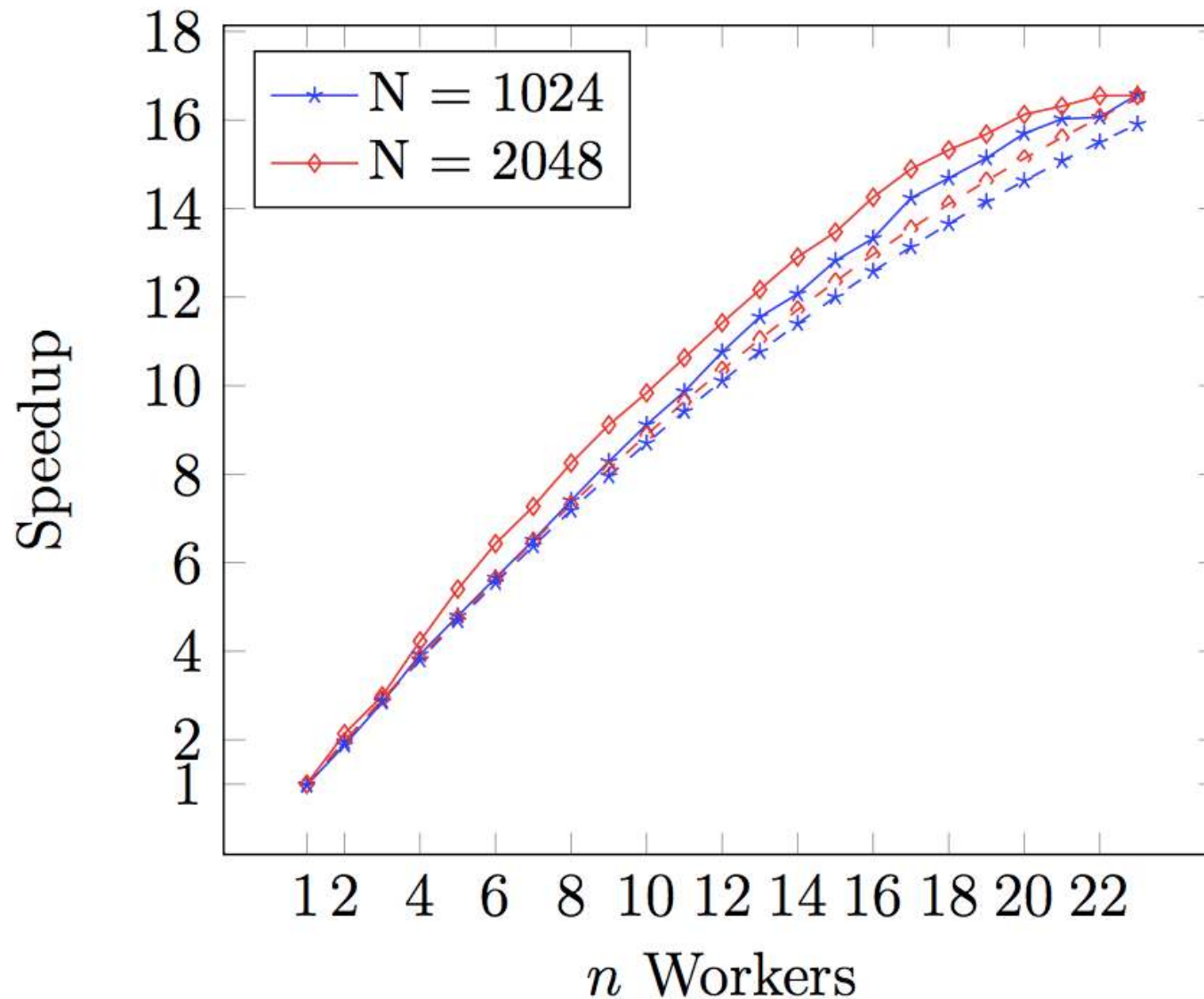
$$\mathcal{T} + \mathcal{T}_{\text{dequeue}}(Q_0) + \mathcal{T}_{\text{enqueue}}(Q_1).$$

If $\mathcal{P}$ produces p number of outputs, then the task farm produces $n \times p$ number of outputs, so the resulting cost needs to be divided by $n \times p/p$, or n :

$$\frac{\mathcal{T} + \mathcal{T}_{\text{dequeue}}(Q_0) + \mathcal{T}_{\text{enqueue}}(Q_1)}{n}.$$

Predicting Parallel Execution Costs

$\text{FARM}_n (\text{FUN } \sigma)$



Matrix multiplication,
 $N \times N$ matrices
24-core AMD Opteron

Predicting Parallel Execution Costs

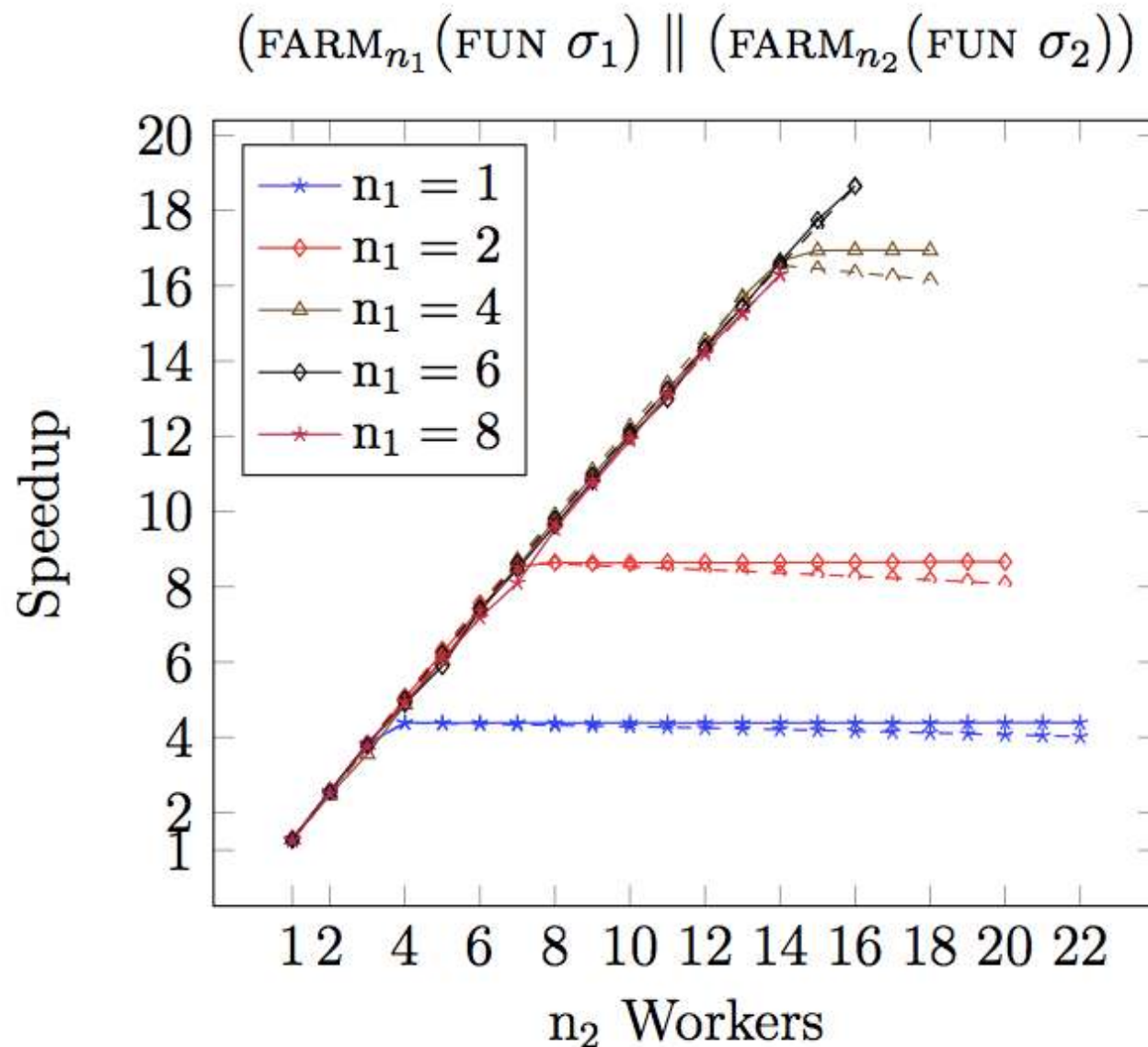


Image Convolution
24-core AMD Opteron

Alternative Parallel Structures

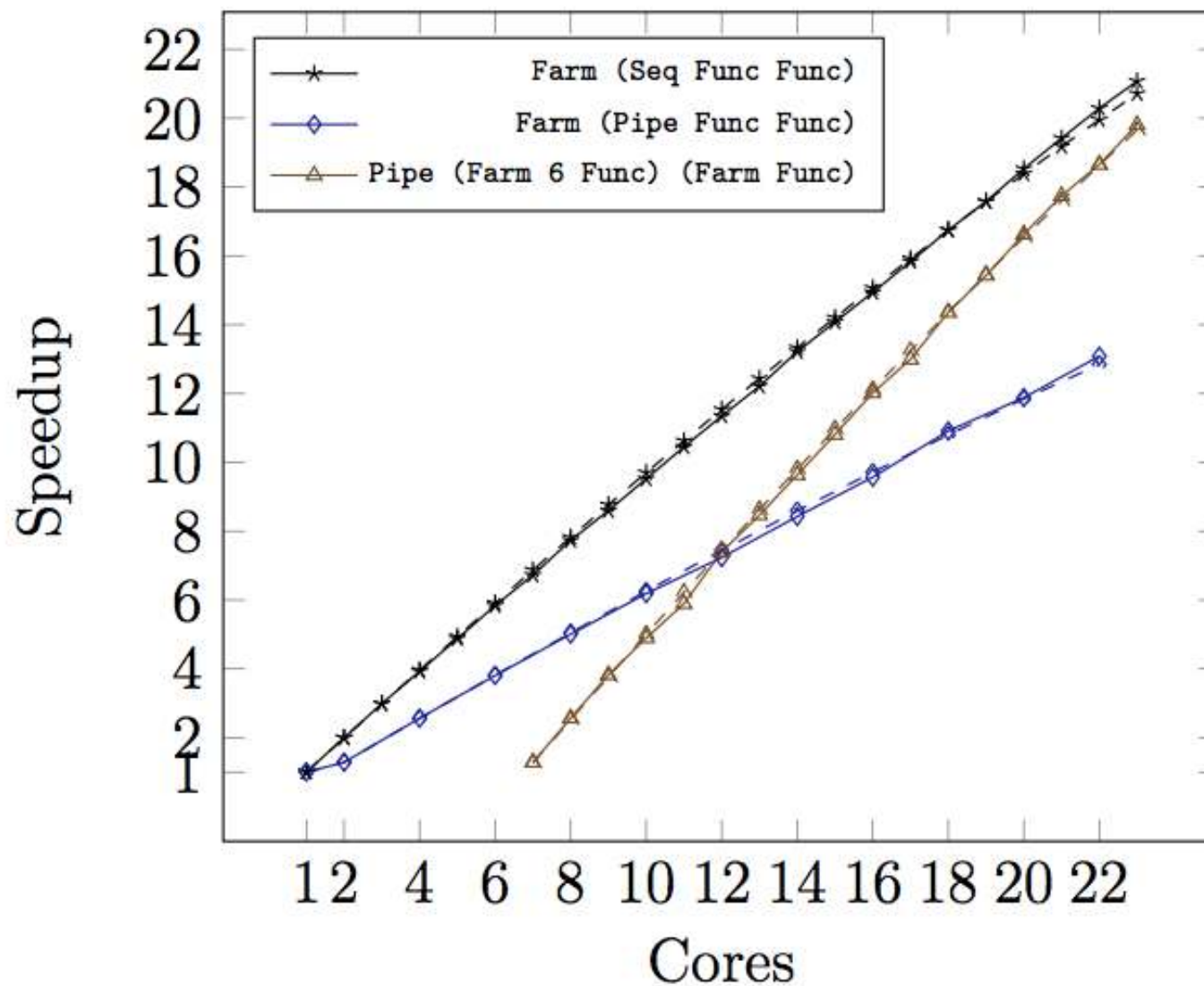


Image Convolution
24-core AMD Opteron

Alternative Parallel Structures

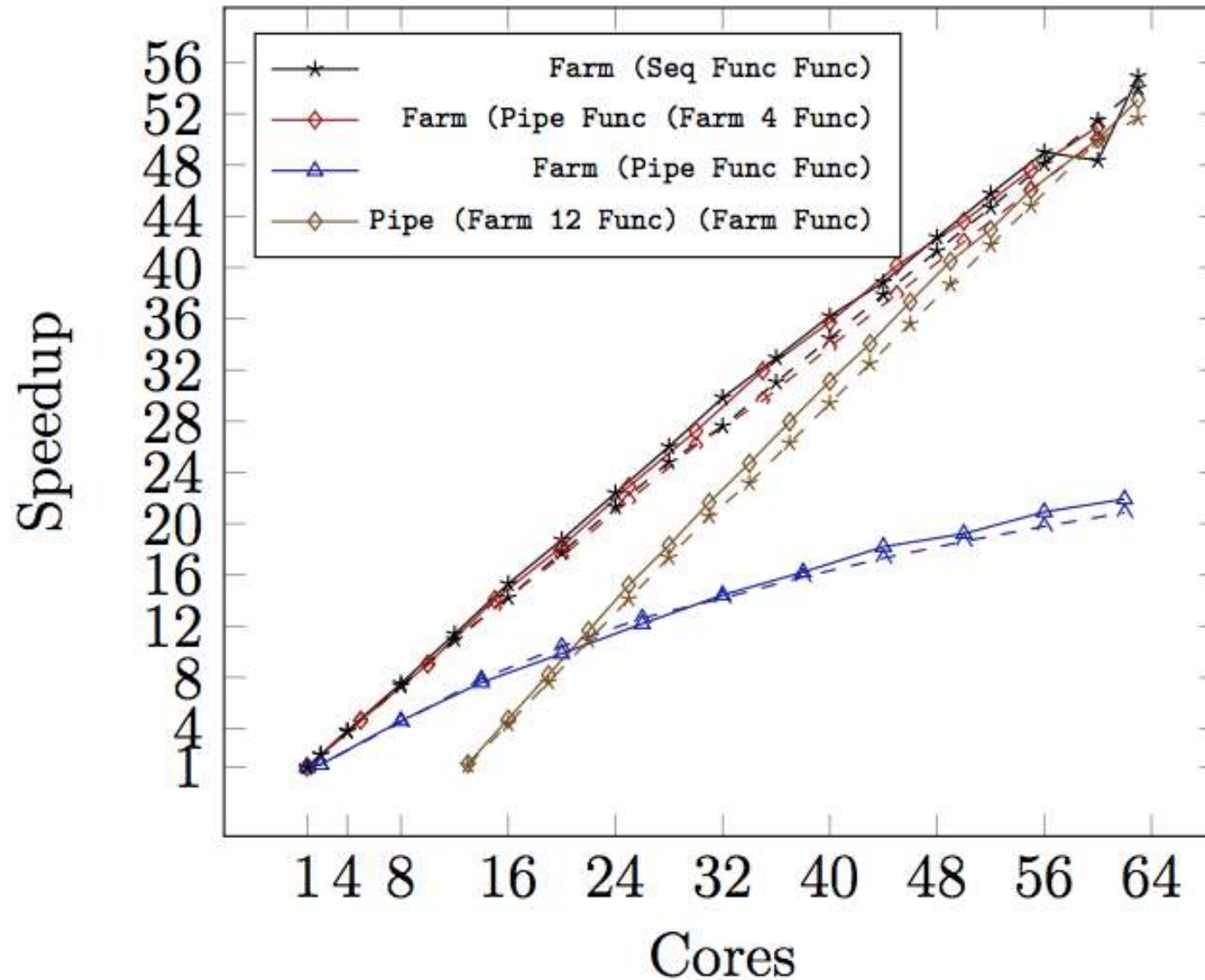


Image Convolution
64-core Intel Xeon

Conclusions

- Deriving costs of parallel structures from an operational semantics is very powerful:
 - **Automatically** derive a **cost equation** from an “implementation”
 - Compile-time information about run-time behaviour based on a simple and easy to understand model.
 - When combined with our previous work (ICFP 2016), we can **automatically** rewrite programs to **minimize costs**
- Our cost model accurately predicts lower bounds on speedups
- We can choose between alternative parallel implementations
 - different patterns
 - CPU/GPU, manycore/multicore

Future Work

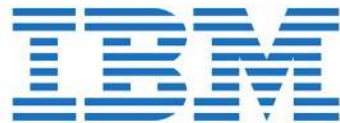
- Other patterns, e.g. stencil and bulk synchronous parallelism
- More general recursion patterns:
 - e.g. adjoint folds or conjugate hylomorphisms (Hinze)
- Apply to real languages (e.g. Haskell, Erlang)
 - Build a full implementation

RePhrase Project: Refactoring Parallel Heterogeneous Software – a Software Engineering Approach (ICT-644235), 2015-2018, €3.5M budget

8 Partners, 6 European countries

UK, Spain, Italy, Austria, Hungary, Israel

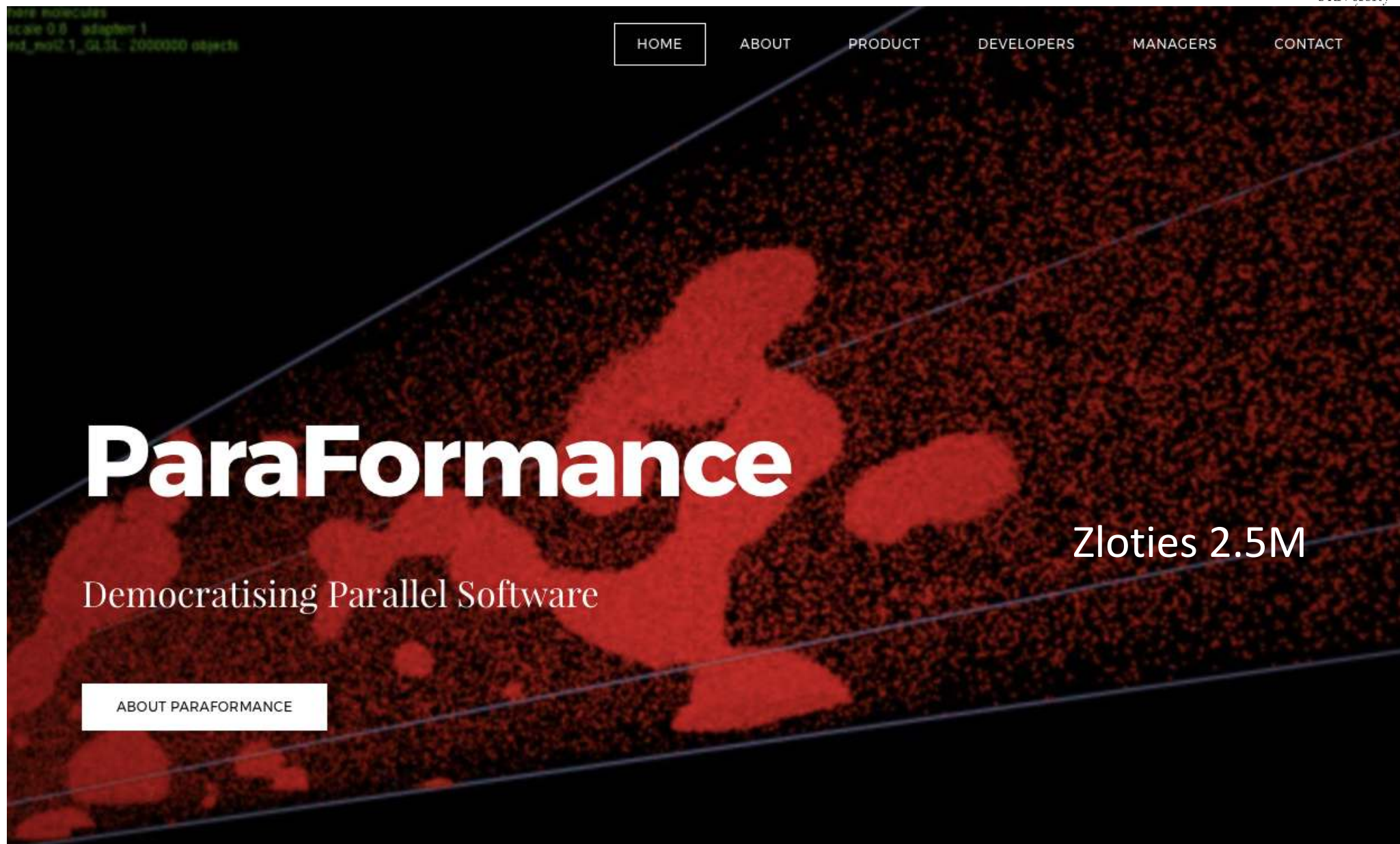
Coordinated by @khstandrews



Universidad
Carlos III de Madrid



Bringing Functional Ideas to the Masses!



THANK YOU!

<http://rephrase-ict.eu>

<http://paraphrase-ict.eu>

@rephrase_eu

Type System

$$\begin{array}{c}
 \frac{\rho(f) = A \rightarrow B}{\vdash f : A \xrightarrow{A} B} \quad
 \frac{\vdash e_1 : B \xrightarrow{\sigma_1} C \quad \vdash e_2 : A \xrightarrow{\sigma_2} B}{\vdash e_1 \circ e_2 : A \xrightarrow{\sigma_1 \circ \sigma_2} C} \quad
 \frac{\vdash e_1 : F B \xrightarrow{\sigma_1} B \quad \vdash e_2 : A \xrightarrow{\sigma_2} F A \quad G = \text{base } F}{\vdash \text{hylo}_F e_1 e_2 : A \xrightarrow{\text{HYLO}_G \sigma_1 \sigma_2} B} \quad
 \frac{\vdash p : T A \xrightarrow{\sigma} T B \quad F = \text{base } T}{\vdash \text{par}_T p : T A \xrightarrow{\text{PAR}_F \sigma} T B}
 \end{array}$$

Figure 5: Structure-Annotated Type System for E .

$$\begin{array}{c}
 \frac{\vdash s : A \xrightarrow{\sigma} B}{\vdash \text{fun } s : T A \xrightarrow{\text{FUN } \sigma} T B} \quad
 \frac{\vdash s_1 : F B \xrightarrow{\sigma_1} B \quad \vdash s_2 : A \xrightarrow{\sigma_2} F A \quad G = \text{base } F}{\vdash \text{dc}_{n,F} s_1 s_2 : T A \xrightarrow{\text{DC}_{n,G} \sigma_1 \sigma_2} T B} \\
 \frac{n : \mathbb{N} \quad \vdash p : T A \xrightarrow{\sigma} T B}{\vdash \text{farm } n p : T A \xrightarrow{\text{FARM}_n \sigma} T B} \quad
 \frac{\vdash p_1 : T A \xrightarrow{\sigma_1} T B \quad \vdash p_2 : T B \xrightarrow{\sigma_2} T C}{\vdash p_1 \parallel p_2 : T A \xrightarrow{\sigma_1 \parallel \sigma_2} T C} \quad
 \frac{\vdash p : T A \xrightarrow{\sigma} T (A + B)}{\vdash \text{fb } p : T A \xrightarrow{\text{FB } \sigma} T B}
 \end{array}$$

- $\sigma \sim A \rightarrow B$ is an alternative notation for $A \xrightarrow{\sigma} B$

$$\frac{\vdash e : A \xrightarrow{\sigma_1} B \quad \sigma_1 \cong \sigma_2}{\vdash e : A \xrightarrow{\sigma_2} B}$$